



LIPPEN: A Lightweight In-Place Pointer Encryption Architecture for Pointer Integrity

Erfan Iravani
Virginia Tech
erfani@vt.edu

Lalit Prasad Peri
Virginia Tech
lalitprasad@vt.edu

Mohannad Ismail
Virginia Tech
imohannad@vt.edu

Charitha Tumkur Siddalingaradhya
Virginia Tech
charitha24@vt.edu

Changwoo Min
Igalia
changwoo@igalia.com

Elif Bilge Kavun
Barkhausen Institut&TU Dresden
elif.kavun@barkhauseninstitut.org

Wenjie Xiong
Virginia Tech
wenjiex@vt.edu

Abstract—Memory-safety vulnerabilities in C and C++ programs continue to enable sophisticated exploitations such as control-flow hijacking and data-oriented attacks. Existing hardware defenses either rely on address space layout randomization or explicit metadata to pointers to verify the integrity. Many mitigation techniques introduce external metadata for security checks, providing strong guarantees, but incur additional memory accesses and memory footprint overhead. In-place protection schemes, such as ARM Pointer Authentication (PAC), achieve low overhead but have limited entropy for security and are susceptible to brute-force attacks.

This paper presents LIPPEN, a hardware–software co-design for *full-pointer encryption* that provides strong pointer integrity without metadata. LIPPEN encrypts every pointer with a block cipher, cryptographically binding it to its execution context, and decrypts it transparently at dereference time. By re-purposing the entire 64-bit pointer field for encryption rather than preserving raw address bits, LIPPEN maximizes entropy, eliminates the brute-force weaknesses of truncated authentication codes, and maintains binary compatibility with existing PAC-enabled software protection. We prototype LIPPEN on FPGA using 64-bit RISC-V Rocket and BOOM cores, and evaluate it with microbenchmarks, nbench, and SPEC CPU2017. We compare against both an in-house PAC implementation on RISC-V and Apple’s PAC on the M1 processor. Across these workloads, LIPPEN provides comprehensive pointer protection with comparable or better runtime overhead to PAC-based schemes, while incurring negligible area and power overhead. These results show that LIPPEN is a practical design point for deploying strong pointer protection in real processors.

I. INTRODUCTION

Modern software systems remain vulnerable to increasingly sophisticated memory corruption exploits. Among the most prevalent and powerful are *control-flow hijacking* and *data-oriented* attacks, which continue to endanger critical infrastructure—from kernels and hypervisors to browsers and database engines. Control-flow hijacking exploits corrupted code pointers such as return addresses, function pointers, or virtual table entries to redirect execution toward attacker-controlled instructions or gadgets. In contrast, data-oriented attacks manipulate data pointers and non-control data to steer legitimate computations toward malicious outcomes without violating the program’s control-flow graph. Together, these attack classes enable arbitrary code execution, privilege escalation, and logic subversion even in hardened environments.

Recent research has explored a wide range of defenses against the pointer forgery attacks. Broadly, these efforts fall into two categories: *address space layout randomization* and *metadata augmentation for integrity*. Address space layout randomization schemes, such as ASLR [83], introduce randomness to memory layouts. However, these defenses rely on a single random offset for each domain’s address space; once the offset is disclosed, the protection collapses.

Alternatively, metadata can serve either as a cryptographic Message Authentication Code (MAC) checked on pointer use [62], [67], or as a capability/permission that prevents unauthorized pointer modification [58], [92], [94]. Most defenses [67], [92], [94] store this metadata in auxiliary structures, providing strong protection but incurring additional memory accesses, memory overhead, and added hardware/ISA management complexity. In contrast, in-place schemes [58], [62] reuse the 64-bit word containing the pointer itself, eliminating metadata memory overhead. A prominent realization of in-place pointer integrity protection is the *Pointer Authentication Code* (PAC) scheme introduced in ARMv8.3-A and later architectures [75]. PAC leverages the 7-16 unused bits of 64-bit virtual addresses to store a MAC alongside the pointer [57], incurring no additional memory overhead [62]. It uses a lightweight block cipher to generate a short MAC over a pointer, combining a secret key with a contextual modifier. On dereference, hardware verifies the MAC before use, preventing straightforward pointer corruption and cross-context pointer reuse. PAC has been extensively leveraged in the research community to enforce control-flow integrity, type safety, and memory safety [40], [49], [53], [59], [62], serving as the foundation for numerous hardware-assisted pointer-protection schemes. In practice, PAC is widely deployed across Arm-based systems to protect return addresses, function pointers, and virtual tables. It secures kernel and user-space control flow on platforms such as Apple *arm64e*, Android, and Windows on Arm [5], [7], [64], [68], [89]. These uses make PAC one of the most prevalent hardware mechanisms for enforcing pointer integrity in both commodity and experimental systems.

However, for in-place schemes [11], [58], [75], because the authentication code must fit within the 64-bit pointer, the effective entropy is usually small (e.g., <24 bits [58]), making

brute-force guessing of valid codes feasible for attackers [52], [70]. A pointer has 64 bits, and thus, in theory, a protection scheme can raise the brute-force space to 2^{64} without additional memory. In the meantime, the information of the pointer value should still be stored in the 64 bits to be a functional pointer. For example, in PAC, pointer values are directly kept, resulting in a small number of bits for authentication and thus a small brute-force space. On the other hand, although the raw pointer values are kept, it is not used directly without authentication in practice.

We propose LIPPEN, an architecture that uses a lightweight block cipher in the Electronic Code Book (ECB) mode to *fully* encrypt the 64-bit pointers for integrity protection. By replacing each 64-bit pointer with its encrypted representation, the architecture can still retrieve the original pointer value when dereferenced. Fully encrypting the pointer significantly improves the entropy limitation due to the limited available unused bits and prevents attackers from forging valid pointer values through brute-force attacks. Still, LIPPEN’s encryption primitive and ISA design provide a similar programming interface to Arm PAC, supporting all protection policies built upon PAC.

In general, encryption alone does not guarantee integrity. Prior work [33], [65] proposed counter (CTR)-mode encryption for pointer protection, but CTR remains vulnerable to targeted bit-flip attacks. C3 [58] uses partial pointer encryption in ECB mode to detect corrupted pointers, but has a limited entropy. To the best of our knowledge, we are the first to evaluate the security and performance of fully encrypting the 64-bit pointer for integrity.

Although encryption and decryption introduce latency overhead, lightweight block ciphers such as PRINCEv2 [24] make full pointer encryption practical with a smaller performance overhead than Arm PAC. Compared to MAC-based authentication like PAC, full-pointer encryption delivers much stronger integrity guarantees in both conventional and transient execution, closing the brute-force gap inherent in in-place authentication codes while preserving PAC’s compatibility advantages. We make the following contributions:

- **Design of LIPPEN.** We propose LIPPEN, a cryptography-based pointer encryption architecture that provides brute-force-resilient protection with lower latency overhead than existing pointer authentication schemes. LIPPEN introduces a compatible ISA that leverages existing PAC compiler infrastructure across protection policies.
- **Co-design system and cipher, and security analysis.** By considering how contextual modifiers are used in practice, we co-design the system and the cipher to avoid using a more expensive tweakable cipher while still providing enough modifier bits. We formally show that if an attacker were able to forge a valid encrypted pointer, one could construct a proxy adversary capable of launching an attack on the underlying block cipher.
- **Implementation and evaluation.** We implement LIPPEN and a baseline PAC design on RISC-V Rocket and BOOM cores on an FPGA prototype, and additionally

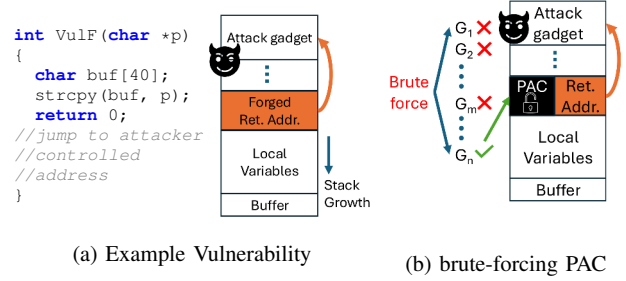


Fig. 1: Memory safety vulnerabilities and PAC.

examine Apple M1 as a real-world PAC deployment. We evaluate using targeted microbenchmarks, nbench, and SPEC CPU2017, analyzing data-pointer and return-address protection, speculation effects, and compatibility with prior PAC-based compiler passes. Our results show that LIPPEN achieves performance comparable to or better than PAC while providing substantially stronger security guarantees. Our implementation and compiler are open-sourced at <https://github.com/bearhw/LIPPEN>.

II. BACKGROUND

A. Control Flow Hijacking and Data-Oriented Attacks

C and C++ underpin kernels, hypervisors, browsers, and high-performance libraries since they offer tight control over layout and performance. The same low-level control, however, exposes programs to *memory safety* violations: *spatial* errors (out-of-bounds reads/writes, type confusion) and *temporal* errors (use-after-free, double free, dangling pointers). These defects arise from unchecked pointer arithmetic, manual lifetime management, and implicit casts, and they commonly yield *arbitrary read/write* primitives after exploitation [31], [74].

Once an attacker can corrupt memory, they can launch *control-flow hijacking*—diverting the program’s execution to attacker-chosen code or gadgets. Classic stack-based overflows overwrite return addresses or saved frame pointers (*stack smashing*) [20], as shown in Figure 1a, while heap-based corruptions target function pointers, C++ vtable pointers, longjmp buffers, PLT entries, or indirect branch targets [50]. Modern exploits prefer code reuse, chaining short instruction sequences to build *Return/Jump/Call*-oriented programming payloads [21], [30], [80], [82], yielding full control of execution without code injection. Memory-safety bugs thus routinely evolve into pointer-forgery primitives that enable both classic and modern exploitation techniques. To counter these attacks, Control-Flow Integrity (CFI) [2] was proposed to ensure that execution follows only legitimate paths derived from the program’s control-flow graph, preventing hijacking through corrupted control data such as return addresses or function pointers. However, numerous bypasses have been demonstrated. Counterfeit Object-Oriented Programming (COOP) [80] and other CFI-bypass attacks [34] show how virtual-table and object-pointer corruption can subvert C++ dispatch even when coarse-grained CFI is present, using techniques such as

control-flow bending [29]. More advanced attacks, including Control Jujutsu [39], NEWTON [90], and AOOR [77], exploit dynamic analysis to bypass even fine-grained CFI protections.

Even when direct control flow is guarded, attackers can compromise program behavior through *data-oriented* and *data-flow* attacks [32], [46]. Data-Oriented Programming (DOP) [46] shows that corrupting data pointers or non-control data can achieve powerful, semantics-preserving computation without altering branch targets. These works show that protecting only control-flow or data values in isolation is insufficient. To substantially raise the bar against modern exploitation, defenders must ensure the integrity of both *code pointers* and *data pointers*, providing comprehensive protection against control-flow hijacking and data-oriented attacks.

Recent advances in memory-safe languages such as *Rust* significantly reduce vulnerabilities by enforcing strong ownership and lifetime semantics at compile time. However, *Rust* cannot eliminate all memory-safety risks: interoperability with legacy C/C++ code, the use of `unsafe` blocks, and low-level system interfaces can still reintroduce pointer corruption. Moreover, large existing software ecosystems written in C and C++ cannot be easily rewritten in *Rust*. Consequently, complementary hardware mechanisms that ensure pointer integrity remain essential to securing modern systems end-to-end.

B. Pointer Authentication Code

Pointer Authentication (PA), introduced in Armv8.3-A, embeds a cryptographic Pointer Authentication Code (PAC) in the unused high-order bits of 64-bit pointers [25], computed from the *pointer*, a 64-bit *modifier* (context), and a *secret key* via dedicated instructions (e.g., `PACIA` to sign, `AUTIA` to verify). On authentication failure, indicating illicit pointer modification, the architecture either corrupts the pointer's top bits to render it unusable (Armv8.3 [5]) or raises a synchronous exception (Armv8.6 [1]).

By signing return addresses and function pointers, PA thwarts control-flow hijacking attacks such as ROP and JOP, and significantly raises the bar against data-oriented exploits, as attackers must forge valid PAC values to hijack control or data pointers [8].

PAC has served as the foundation for numerous research prototypes that extend pointer authentication to enforce broader security properties such as type safety, temporal safety, and data integrity. Examples include PARTS [62], which introduces PAC-based control-flow integrity; PACStack [61], which chains authenticated return addresses to strengthen backward-edge protection; PTAAuth [40], which dynamically detects temporal memory corruptions; PACMem [59], which unifies spatial and temporal protection; AOS [53], which combines PAC with architectural metadata for object safety; PacTight [49], which prevents misuse of security-sensitive pointers through unique modifiers; and RSTI [48], which systematically enforces C scope and type at runtime. Beyond research prototypes, PAC has been widely adopted across Arm-based platforms to harden both kernel and user space. Apple's *arm64e* architecture uses a customized version of

PAC to sign return addresses, function pointers, and C++ virtual tables, with compiler and hardware support integrated into iOS, macOS, and their system libraries [7], [64]. Linux and Windows on Arm similarly deploy PAC to protect kernel control-flow structures and sensitive function pointers [68], [89]. The Clang/LLVM toolchain provides first-class support for emitting PAC instructions (`PACIASP`, `AUTIASP`, `BLRAA`) and manages key usage transparently during code generation [64]. Collectively, these experimental and commodity systems demonstrate PAC's versatility as a hardware-assisted substrate for enforcing comprehensive pointer integrity.

Brute-force Attacks on PAC. Due to the limited PAC size, an attacker can mount a brute-force attack by exhaustively trying all possible authentication codes (e.g., $2^{16} = 65,536$ candidates [76], reduced further by 4 bits when combined with Memory Tagging Extension (MTE) [11]), as illustrated in Figure 1b. Such attacks can proceed both conventionally and speculatively. In a modern processor, instructions, including pointer authentication operations, may execute along mis-speculated paths. If a forged pointer with an incorrect PAC is speculatively used, no architectural fault is raised; instead, the subsequent memory loads fail because authentication has failed. Consequently, the act of speculatively executing pointer authentication and using the resulting pointer can leave detectable microarchitectural effects (for instance, a cache access might occur only if the PAC was correct) [76]. This creates a potential PAC oracle: a side channel through which an attacker can learn whether a guessed PAC was valid or not, by observing subtle hardware state changes, all without triggering any program-visible error. The PACMAN study [76] demonstrated that speculative execution can be exploited to create PAC oracles, enabling systematic brute-forcing of PAC values within a reasonable time (approximately 2.94 minutes on an Apple M1 chip), effectively removing the primary barrier to control-flow hijacking on PAC-protected platforms.

For mitigation, Arm recommends PAC clearing via `XPAC` immediately after authentication, e.g., `AUT`; `XPAC`, which eliminates residual PAC state and prevents speculative dereference of corrupted pointers. In later versions, Arm introduces `FEAT_FPACC_SPEC`, making the architectural states not measurably different between failing or passing the authentication [10]. However, these fixes mean the forged pointers can be used during transient execution, not protecting the pointers in Spectre attacks [42], [54], [70]. Attackers can then launch attacks like speculative ROP [54].

III. THREAT MODEL

We follow a threat model of typical memory-corruption attacks [67] and its speculative version [54]. The attacker aims to exploit memory-safety vulnerabilities, such as stack buffer overflows and use-after-free bugs, to corrupt pointers and thereby subvert control flow or mount data-oriented attacks. We assume a powerful attacker who, after exploiting such a vulnerability, can perform arbitrary reads of process memory and overwrite any writable memory location, including code pointers, data pointers, and user-space protection metadata

TABLE I: Comparison of representative pointer integrity defenses.

Category	Defense	Brute-force Resilience	Memory Footprint Overhead	Required Modification	# Context / pointer within domain
Address Randomization	ASLR [83]	19–28 bits ^a	None	OS	No
	Morpheus [41]	60 bits ^a	2 bits/word	Crypto Engine + Churn-Unit + OS	No
External Metadata	CHERI [92]	Metadata not accessible in user space. Cannot be brute forced	256 bits/pointer	Instructions + compiler + memory hierarchy	Included in tags
	ZeRØ [94]		2 bits/word		
	PUMP [37]		word-size bits/word		
	Star [43]		(2,6) bits/word for (Data, Instructions)		
	CCFI [67]	128 bits	128 bits/pointer	Compiler ^b	80 bits
In-pointer Metadata	FRP [73]	52 bits	16 bytes/object	OS + memory hierarchy	No
	PAC [62]	7–16 bits	None	Instructions + Compiler ^c + Hardware Crypto Engine	64 bits
	C3 [58]	24 bits			Yes
	LIPPEN (ours)	64 bits			Adjustable

^a This entropy accounts for all the addresses, not per pointer. i.e., a successful brute-force will break all pointers, not just for one pointer.

^b They use Intel AES-NI engine for encryption.

^c Our work (LIPPEN) uses the same Instructions and Compiler support as Arm PAC.

such as modifiers. This capability enables both control-flow hijacking and data-oriented manipulation. We also include pointer forgery during transient execution attacks [42], [54], [70], i.e., pointer integrity is violated during speculation, e.g., Speculative ROP [54], or forging a pointer in Spectre attacks. However, we focus on pointer integrity protection; other information leakages due to side channels not exploiting pointer integrity are out of scope.

We trust the underlying hardware and operating system, which are responsible for securely generating, managing, and storing the process-wide secret key K , which remains constant during execution and is inaccessible to user-level code.

IV. LIPPEN DESIGN

A. Design Goals

LIPPEN is designed to ensure robust pointer integrity while maintaining practicality and efficiency in both conventional and speculative execution. The key design goals are:

- G1: Comprehensive Pointer Integrity Coverage.** Protect all pointer types, including both data pointers and code pointers (e.g., return addresses).
- G2: Zero Metadata Overhead.** Eliminate auxiliary data structures, shadow memory, or tag tables to avoid memory overhead and access latency.
- G3: High Security Strength.** Provide strong protection against pointer corruption, reuse, and brute-force attacks by maximizing cryptographic entropy within the pointer representation.
- G4: Low Performance Overhead.** Use lightweight, hardware-assisted encryption to achieve near-PAC runtime overhead, enabling deployment in performance-sensitive systems.
- G5: Flexible Hardware Support for Various Security Policies.** Support multiple protection policies and context-binding schemes under user or compiler control, e.g., modifier in PAC, enabling flexible trade-offs between performance, security, and compatibility.

G6: Reuse of the Existing Toolchain for Easy Deployment of Defenses

Preserve existing compiler instrumentation, Application Binary Interface (ABI) conventions, and runtime interfaces so that PAC-enabled software can run on LIPPEN without much modification.

B. Design Space Discussion

A wide range of schemes has been developed to defend against memory corruption and control-flow attacks. Table I compares representative designs across four axes: brute-force space, memory footprint overhead, deployment requirements, and context granularity within each domain. We analyze their trade-offs and how they align with our design goals. Broadly, these schemes fall into two primary categories: *address layout randomization* and *metadata augmentation*.

a) Address Randomization: Address-randomization defenses [41], [84] increase attacker uncertainty by randomizing the placement or representation of code and data objects. Conventional ASLR is practical and widely deployed because it requires no changes to program binaries or pointer formats, but it provides only probabilistic protection: once layout entropy is disclosed or guessed, leaked pointers can be reused to construct code-reuse or control-flow hijacking attacks. Prior work [70] further shows that such secret offsets can be inferred through speculative probing attacks [42]. Morpheus [41] strengthens this class of defenses with hardware-supported moving-target defenses and runtime churn. It applies a random displacement to code and data pointers by adding a secret offset to their values, and periodically changes these offsets during execution (e.g., every 50ms in the evaluated configuration) so leaked or brute-forced information becomes stale. This stronger protection, however, requires substantial architectural support, including 2-bit runtime domain tags per 64-bit word, additional tag storage/cache structures, and specialized hardware for churning, pointer translation, and attack detection. Thus, while address randomization is an efficient baseline defense, conventional ASLR does not provide high security level (**G3**), and stronger variants such as Morpheus achieve higher security only with nontrivial

hardware and metadata complexity, falling short of our goal for zero metadata overhead (**G2**). Additionally, because the offsets are shared by many pointers, it cannot support adding context for each individual pointer (**G5**).

b) Metadata Augmentation: A second family of techniques strengthens pointer integrity by associating pointers with auxiliary integrity or capability metadata [33], [43], [58], [62], [67], [73], [78], [92], [94]. Depending on where metadata is maintained, these defenses can be grouped into two categories:

Protection with external auxiliary metadata structures. Systems such as CCFI [67], ZeRØ [94], Star [43], and capability-based architectures like CHERI [92] associate pointers with auxiliary metadata structures or tagged memory that encode authentication codes, bounds, or permissions. Such schemes provide robust protection, comprehensive coverage, and strong security guarantees by maintaining precise, per-pointer metadata. However, they incur nontrivial memory lookup and synchronization overheads. Their reliance on external structures complicates hardware design and violates the **zero-metadata** principle (**G2**), making them less practical for lightweight or commodity environments.

In-Place Protection. Several works, such as Arm Pointer Authentication (PAC) [25] and C3 [58], embed integrity metadata directly into the pointer representation. Such approaches have zero metadata overhead and are adopted by industry [25]. However, subsequent analyses have demonstrated that such schemes are vulnerable to brute-force or collision-based attacks (**G3**): PAC is compromised by PACMAN [76], and C3 is bypassed by Na *et al.* [70]. The underlying reason is that the number of authentication bits embedded within the pointer is limited; PAC uses only 7–16 bits, and C3 employs a 24-bit cipher, significantly constraining the available integrity space. Similarly, FRP [73] encodes 52 bits of the pointer, including the unused high order bits; however, its encoding/decoding uses table (map) lookup instead of cryptography, introducing extra indirection and performance overhead (**G4**). Without new instructions, it relies on malloc to manage the map and supports only heap objects.

Arm Pointer Authentication. Arm PAC [62], retains the raw pointer address and reserves only the unused bits for the authentication code. This design choice stems from two considerations by Arm [75]: (i) preserving the full address value allows pointers to participate in branch prediction without authentication, and (ii) maintaining meaningful address values simplifies software debugging and crash analysis. These choices, however, dramatically reduce the number of bits available for authentication to <16 bits [57], depending on the virtual address space size.

In practice, the pointer value in the PAC-protected pointers is not used until authentication strips the PAC. Arithmetic on the raw pointer value without stripping the PAC will also corrupt the authentication state and cause subsequent checks to fail. One potential benefit of keeping the raw pointer value is that it leaves room for micro-architectural optimizations, such as using the pointer speculatively before

the authentication completes, hiding the authentication latency. e.g., Arm also provides the fused instruction for authentication and load (LDRAA and LDRAB) and authentication and return (RETAA and RETAB). However, in practice, as shown in the PACMAN [76] attack, the authentication result impacts the execution result of the follow-up instruction consuming the pointer, indicating limited overlap between the load or return operation and authentication. Otherwise, if a raw data pointer value is used speculatively before authentication, the TLB must be updated regardless of the authentication result, since it lies on the critical path. This would prevent the PACMAN attack from succeeding on data pointers. Additionally, PARTS [62] uses additional instructions to emulate the 4-cycle authentication delay for end-to-end performance evaluation, showing reasonable performance without speculation.

Arm also provides the XPAC instruction set (e.g., XPACI, XPACD, and XPACLRI) to strip the PAC from a pointer and recover its original address [13]. However, XPAC operations are typically invoked only in specialized contexts where security is not a concern, such as specific pointer arithmetic, low-level runtime code, or debugging, and are rarely executed in normal application paths.

This paper: Full Pointer Encryption. These findings suggest that preserving full raw address bits in authenticated pointers may not be necessary in practice while severely constraining available entropy. LIPPEN leverages this insight to address the core limitations of prior designs. By repurposing all 64 bits of the pointer for cryptographic protection, LIPPEN achieves comprehensive coverage across all pointer types (**G1**), eliminates external metadata to meet the zero-overhead requirement (**G2**), and maximizes entropy to provide strong cryptographic protection against brute-force and reuse attacks (**G3**). This full-pointer encryption approach removes the entropy bottleneck inherent to truncated MACs and transparently restores a valid address upon dereference, providing robust integrity and confidentiality without additional memory or hardware state. However, full pointer encryption still faces challenges in finding a suitable cipher, conducting a security evaluation, and providing debugging support when needed.

C. Choice of Cipher

A central requirement in LIPPEN’s design is to achieve strong cryptographic protection with minimal performance impact (**G4**). As pointer unsealing and sealing occur frequently along critical execution paths, the encryption primitive must be “light” enough in terms of operation latency, area, and power. Since LIPPEN aims to protect the pointers in 64-bit machines without adding additional memory footprint (**G2**), we consider lightweight block ciphers that operate on a message size of 64-bit or less. Table II summarizes the intended design goals, security levels (key sizes), block sizes, and expected latency and area characteristics of the widely-used ciphers among these proposals.

Early proposals such as KATAN and KTANTAN [28], LED [45], Piccolo [85], TWINE [88], and KLEIN [44] demonstrate this direction of design clearly. Prominent lightweight

TABLE II: Lightweight cipher candidates for LIPPEN.

Cipher	Block Size (bits)	Key (bits)	Tweak (bits)	Rounds	Area (GE) ^a	Latency (cycles ^b or ps)	Notes relevant for LIPPEN
<i>Early lightweight / area-optimized ciphers (not latency-optimized, area = $\sim$ listed area cost $\times$ no. of rounds)</i>							
KATAN/KTANTAN [28]	64	80	—	254	$\sim 3200/\sim 3000$	~ 254 cyc	Bit-serial, ultra-low-area, long latency.
LED-64 [45]	64	64/128	—	32/48	~ 1200	32 cyc	Compact SPN; unrolled path too deep.
Piccolo-80/128 [85]	64	80/128	—	25/31	683/758	432/528 cyc	Very small area; serialized 4-bit datapath.
TWINE-80 [88]	64	80	—	36	1503	36 cyc	Round count too large for 1-cycle use.
KLEIN-64/80/96 [44]	64	64/80/96	—	12/16/20	1981/2097/2213	105/107/109 cyc	Compact SPN; Not optimized for unrolling.
PRESENT-80 [22]	64	80	—	31	1570	32 cyc	ISO lightweight cipher; unrolled area large.
SIMON64/128 [18]	64	128	—	44	$\sim 1200\text{--}1400$	44 cyc	Hardware-oriented Feistel; too many rounds.
ASCON (perm.) [38]	320 (rate 64/128)	128	—	6/8 perms	27280	6 cyc	AEAD permutation; not a 64-bit block cipher.
<i>Low-latency ciphers (suitable candidates for pointer authentication, e: encryption, d: decryption)</i>							
K-Cipher [55] ^c	var.	param.	—	10–14	42552(e)	767ps/2–3 cyc	High-throughput low-latency via pipelining.
BipBip [19] ^c	24	128	128	7 (stages)	5741(d)	622(d) ps/3 cyc	Depth-3 pipeline; block size too small.
QARMA-64 [16] ^d	64	128	64	11	22131(e/d)	553 ps	Critical path larger than PRINCE-family.
PRINCE [23] ^d	64	128	—	12 (5+mid+5)	13468(e/d)	401 ps	Designed for single-cycle unrolled latency.
PRINCEv2 [24] ^d	64	128	—	12	14181(e/d)	404 ps	Best latency/area trade-off for LIPPEN.

^a Area values are representative gate equivalents (GE) as reported in the original cipher publications or widely cited hardware implementation studies for compact round-based implementations.

^b We report the cycle counts for typical round-based hardware implementations. For low-latency ciphers, the latency is reported in ps.

^c The numbers for K-Cipher and BipBip are taken from the original publications [19], [55]. Note that we normalized the area result for K-Cipher as GE based on Intel 10 nm library characteristics provided in [15], it is originally reported as $1875\mu\text{m}^2$. We report decryption-only numbers for BipBip as highlighted also in the original work.

^d The results for QARMA, PRINCE, and PRINCEv2 are taken from the original PRINCEv2 work [24], as all ciphers were implemented in NanGate 15 nm technology setting, which provided us with a fairer comparison. Note that, according to our calculations based on NanGate 15 nm Open Cell Library Databook, 13468 GE translates to $3391\mu\text{m}^2$ for PRINCE and 14181 GE translates to $3570\mu\text{m}^2$ for PRINCEv2. We report only the results for e/d shared datapath architectures in the table, encryption-only results are slightly smaller/faster than these.

block ciphers such as the ISO-standard PRESENT [22], the NIST lightweight authenticated encryption standard ASCON [38], and the NSA’s SIMON family [18] follow similar principles. However, while these designs offer small round-based hardware footprints, their large number of rounds renders them impractical for unrolled implementations, where the full datapath is required in a single cycle. Their corresponding critical paths are too long for tightly integrated pointer authentication on the processor’s load–use path (see Table II).

For LIPPEN, which performs a pointer integrity check (i.e., decryption) on all protected pointers before use, the decryption latency directly influences the critical path of the program. This requirement moves the design space away from classical lightweight designs and toward *low-latency* block ciphers explicitly engineered for unrolled hardware implementations. Examples include K-Cipher [55], BipBip [19], QARMA [16], PRINCE [23], and PRINCEv2 [24], all of which target ultra-short critical paths. Designs such as K-Cipher and BipBip offer deeply pipelined low-latency modes (e.g., depth-3 pipeline at around 4–4.5 GHz in 10 nm technology for both ciphers [19], [55]), which are useful for high-throughput applications. However, the security level of BipBip is aligned for encrypting 24-bit blocks in every encryption, which is not suitable for LIPPEN’s encryption of 64-bit blocks requirement. Furthermore, K-Cipher’s parameterizable structure and few rounds do not provide the same level of publicly scrutinized provable security margins as PRINCE-family ciphers [66]. In terms of security level, latency, and area trade-off, PRINCE-family ciphers still perform better. The comparison between ciphers is best reflected in Table 6 presented in the original PRINCEv2

TABLE III: Cipher area and timing in VCU118.

Cipher	LUTs	FFs	Latency (cycles)	Fmax (MHz)
QARMA-vhd [47]	1670	65	2	67
PRINCE-vhd [47]	1233	65	2	84
PRINCE-vhd+mod	1250	65	2	72
QARMA-unrolled-verilog	1794	0	1	40
PRINCE-unrolled-verilog [23]	1378	0	1	41
PRINCEv2-unrolled-verilog	1378	0	1	44
PRINCEv2-unrolled-verilog+mod	1522	0	1	42

cipher manuscript [24], where the authors provide the latency and area results for both PRINCE and PRINCEv2 in NanGate 15 nm Open Cell Library. We summarize the results in Table II. In the best latency setting (401 ps) for PRINCE, the area consumption in μm^2 is more than K-Cipher; however, the area cost for PRINCE with a latency of 600 ps is comparable to K-Cipher [24].

QARMA, PRINCE and PRINCEv2 are explicitly optimized for one-cycle unrolled datapaths under reasonable area budgets. QARMA [17] offers a 64-bit tweak, which can encode context for pointer protection; however, it demands significantly larger area and suffers from longer logic depth in unrolled mode than PRINCE due to the tweak. The PRINCE-family provides strong built-in support for “encryption = decryption” with minimal overhead and constant-time structure design. PRINCEv2 offers a more suitable balance of latency, area, and security margins for pointer integrity. However, without a tweak structure like QARMA, a scheme is needed to incorporate the context for pointer protection.

As the values in Table II are collected from each individual paper and are not normalized across works, it only provides an approximate comparison of design complexity. For the ciphers considered in LIPPEN, we implemented representative designs on an AMD/Xilinx VCU118 FPGA platform with a Virtex UltraScale+ device and report post-implementation area and timing in Table III. We use publicly available HDL implementations as baselines: the QARMA Verilog implementation is adapted from the corresponding VHDL design, while the PRINCE and PRINCEv2 implementations are adapted from [23]. The results show that the unrolled PRINCE-family implementations are the best fit for LIPPEN’s latency and area requirements. Relative to unrolled QARMA, PRINCEv2 requires fewer LUTs and achieves a slightly higher maximum frequency, while maintaining single-cycle latency. The `+mod` variants add the XOR logic needed to add additional context bits into the PRINCE datapath. Although this modification increases area relative to the unmodified PRINCE-family designs, PRINCEv2+`mod` remains substantially smaller and faster than unrolled QARMA.

D. ISA and Programming Interface Design

LIPPEN aims to introduce strong pointer protection, while trying to reuse existing software infrastructures rather than redesigning them. Thus, LIPPEN tries to realize full-pointer encryption in a way that preserves PAC’s practical advantages (compact representation, compiler and ABI compatibility, context binding, and in-line hardware operation) while fundamentally elevating its security guarantees.

In PAC, the use of context (modifier) is essential to achieve **G3** and **G5**: it cryptographically ties each pointer to its context, such as a stack frame, privilege level, or protection domain, so that even if a pointer is leaked or copied, it cannot be validly reused in another context (cross-context pointer reuse attack). In LIPPEN, every pointer is treated as an encrypted capability. The encrypted pointer will be similar PAC-protected pointers in Arm. To achieve **G3** and **G5**, each pointer is encrypted with a *context* when generated or stored and decrypted only at the point of dereference, ensuring integrity and confidentiality throughout its lifetime, as shown in Figure 2.

To achieve **G6** (*Seamless Deployability*), the ISA extension of LIPPEN is very similar to that of PAC, achieving minimal disruption to existing software and toolchains. The system manager or operating system configures protection settings using the `SET_KEY` and `SET_M_SIZE` instructions. Here, `SET_KEY` assigns the 128-bit encryption key to the current security domain, while `SET_M_SIZE` specifies the configurations of the modifier components.

Applications can use the `PTR_SEAL` and `PTR_UNSEAL` instructions, which provide an efficient interface for encrypting and decrypting pointers. Their semantics closely mirror PAC’s `PAC*` and `AUT*` instructions, allowing direct reuse of existing compiler instrumentation, Application Programming Interface (API), and ABI conventions without modification. Table IV

TABLE IV: Pointer Encryption ISA Extensions

Instruction	Description
<code>SET_KEY(K1, K2)</code>	Assigns the 128-bit encryption key by concatenating K_1 and K_2 .
<code>SET_M_SIZE(conf)</code>	Sets the configurations for modifier use.
<code>PTR_SEAL(ptr, mod)</code>	Encrypts a 64-bit pointer using the system-managed secret key and an optional modifier. Invoked when a pointer is created or stored.
<code>PTR_UNSEAL(ptr, mod)</code>	Decrypts and validates an encrypted pointer before dereference.

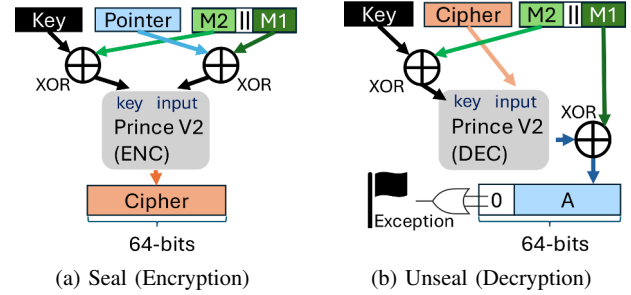


Fig. 2: Pointer encryption and decryption design with PRINCEv2. Key has 128 bits, input/output 64 bits, and M1 and M2 sizes are user defined. After decryption completes, we expect the unused bits of the resulting plaintext to be all zeros, otherwise an exception flag will be raised.

summarizes their functionality. Also, for debugging purposes, the encryption can be turned off.

E. Modifier Design

To address **G3** (*Protection Against Reuse-Based Attacks*), **G5** (*Configurable Protection Modes*), and **G6** (*Reusing Existing PAC Toolchain*), LIPPEN introduces a flexible modifier design that enables fine-grained control over how pointers are cryptographically bound to their execution context. A key principle in pointer encryption is that each *encrypted pointer* must depend on three components: (i) a secret key managed by the system and isolated from software control, (ii) the pointer value itself, and (iii) a context-dependent modifier that ties the encrypted pointer to specific execution conditions, preventing its reuse in unauthorized domains.

1) *Design Challenges*: As shown in Table II and III, PRINCEv2 has the best latency among the 64-bit block ciphers. Ciphers with tweaks incur higher latency and area to process the tweaks. Can we co-design system and cipher to reduce the need for tweaks and lower the protection latency?

2) *Design Approach*: PRINCE encryption has plaintext message and key as the input. So the design options are to mix the modifier into the plaintext or key. Specifically, we define the encryption and decryption as:

$$\text{cipher} = \text{seal}(k, ptr, m) = \text{Enc}_{k \oplus m_2}(\text{plain} \oplus m_1)$$

$$\text{plain } ptr = \text{unseal}(k, \text{cipher}, m) = \text{Dec}_{k \oplus m_2}(\text{cipher}) \oplus m_1$$

Here, $m = m_1 || m_2$ represents modifier components derived from the execution context (e.g., privilege level, address-space

identifier, or control-flow epoch), as illustrated in Figure 2. The lengths of m_1 and m_2 can be configured by the system manager using the `SET_M_SIZE( $m_1$ ,  $m_2$ )` instruction, as described in Table IV. We modified the PRINCE and PRINCEv2 source codes to incorporate the XOR logic for m_1 and m_2 and compared their maximum frequency and area with QARMA, as shown in +mod rows in Table III. PRINCEv2 continues to deliver a better performance.

3) Modifier Design Security Analysis:

a) *Attacker Assumptions:* Based on our threat model, the attacker may exploit software vulnerabilities to overwrite *protected pointer* and *modifier* values at runtime. However, the attacker does *not* have access to or control the key K for any process. We also assume the attacker may have access to a set of observed tuples $\mathcal{T} = \{(m_i, p_i, c_i) \mid c_i = \text{Enc}_K(p_i)\}$ potentially by observing the victim program. The goal of the attacker is to forge an encrypted pointer that dereferences to a *target memory location* p_a , thereby violating pointer integrity. A successful attacker can create a valid tuple (m_a, p_a, c_a) such that c_a decrypts correctly under modifier m_a to an attacker-chosen pointer p_a , i.e., $p_a = \text{Dec}_{K \oplus m_{2,a}}(c_a) \oplus m_{1,a}$.

b) Potential Target Bit Flip Attacks with m_1 :

If m_1 overlaps with the pointer *ptr*, and X denote the bit-mask for the overlapping bits, then the attacker can construct $(m_{1,a} = m_{1,v} \oplus X, p_a = p_v \oplus X, c_a = c_v)$ where (m_v, p_v, c_v) is a valid tuple. As a result, the attacker can overwrite the modifier to flip certain bits in the pointer if X is not zero.

To avoid this ambiguity, m_1 should be placed only in pointer bits that do not affect the address bits. On most 64-bit architectures, only the lower A bits are used as virtual-address bits, e.g., $A = 48$ for a 48-bit virtual address space; the remaining $64 - A$ high-order bits are unused or sign-extension bits. In addition, if pointers are word-aligned, the two least-significant bits do not affect the addressed word. These bits can therefore be safely used for m_1 , yielding $|m_1|_{\max} = 64 - A + 2$.

c) *Potential Key-Collision Attacks with m_2 :* Assume the operating system assigns the victim process a secret key k_v and the attacker process a key k_a . A potential concern is whether an attacker could exploit the m_2 modifier to induce a key collision across protection domains. Specifically, suppose the attacker attempts to construct a relation of the form $k_a = k_v \oplus m_{2,d}$ where $m_{2,d}$ denotes a chosen modifier difference. If such a relation were achievable, the attacker could attempt to craft a tuple $(m_{2,a} = m_{2,d} \oplus m_{2,v}, p_a = p_v, c_a = \text{Enc}_{k_a}(p_v))$, and inject it into the victim domain, thereby forging a valid pointer. We prevent such a collision with our key assignment.

d) *Key Assignment:* To prevent cross-domain key-collision attacks induced by adversarial choices of m_2 , the system enforces that the *unaffected* portion of the domain key is unique across security domains. Concretely, if two domains share the same $128 - |m_2|$ unaffected key bits, then an attacker could choose m_2 values that cause their derived key to match the victim's derived key. Ensuring uniqueness of the unaffected portion eliminates this possibility. This policy also bounds the number of simultaneously supported security domains: since

only $128 - |m_2|$ bits are reserved for domain separation, the maximum number of unique domains is at most $2^{128 - |m_2|}$.

e) Proof of Security:

We define $\text{Adv}_{\text{LIPPEN}}(\mathcal{A})$ denote the advantage of adversary $\mathcal{A}$ in forging an encrypted pointer c_a for chosen (m_a, p_a) for LIPPEN knowing a set of tuples $\mathcal{T}$, and $\text{Adv}_E(\mathcal{B})$ being the advantage of adversary $\mathcal{B}$ in distinguishing the block cipher $E_K(\cdot)$ from a uniformly random permutation P even with oracle access to both $\text{Enc}_{E_K}(\cdot)$ and $\text{Dec}_{E_K}(\cdot)$. For LIPPEN, the implementation ensures that $|m_1|$ uses only unused pointer bits, and the $128 - |m_2|$ key bits are never shared between domains.

Theorem 1. *For any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ knowing $\mathcal{T}$ running in time t , there exists a PPT adversary $\mathcal{B}$ such that $\text{Adv}_{\text{LIPPEN}}(\mathcal{A}) \leq \text{Adv}_E(\mathcal{B}) + \varepsilon(q)$.*

Proof. $\mathcal{B}$ is given oracle access to either a real block cipher $E(\cdot)$ or a random permutation P for both encryption and decryption. $\mathcal{B}$ runs $\mathcal{A}$ by simulating the protocol using its oracle in place of $E(\cdot)$ as follows.

- 1) Answer all of $\mathcal{A}$'s queries to build $\mathcal{T}$ by querying the oracle, i.e., encryption or decryption to obtain c_i with key $k_v \oplus m_{2,i}$ and plaintext $p_i \oplus m_{1,i}$.
- 2) For (m_a, p_a) , run $\mathcal{A}$ to forge pointer c_a . $\mathcal{B}$ queries the oracle to encrypt $\text{plain} = p_a \oplus m_{1,a}$ with key $k_v \oplus m_{2,a}$. $\mathcal{B}$ repeats for q times and if more than $q/2$ encryption of plain matches c_a , $\mathcal{B}$ returns that a real block cipher $E(\cdot)$ is behind the oracle; otherwise, $\mathcal{B}$ returns that a random permutation P is behind the oracle.

If the oracle is for a real block cipher, then the probability of $\mathcal{B}$ returning the right value (i.e., $\text{Adv}_E(\mathcal{B})$) is the probability $\mathcal{A}$ returning the right c_a more than half of the time (i.e., $\text{Adv}_{\text{LIPPEN}}(\mathcal{A})$). If the oracle is for a random permutation P , then $\mathcal{B}$ will return the wrong result if c_a happens to be the output of the random permutation, which is of probability $2^{-64 \times q/2}$. Thus, $\text{Adv}_E(\mathcal{B}) \geq \text{Adv}_{\text{LIPPEN}}(\mathcal{A}) - \varepsilon(q)$ $\square$

Thus, breaking LIPPEN is not easier than breaking the underlying block cipher. The security of the scheme therefore reduces to the cryptographic strength of the underlying PRINCE-family cipher. Existing cryptanalysis of the PRINCE family primarily targets reduced-round variants or relies on implementation attacks such as differential fault analysis [3], [35], [51], [69], [86], [87] and has not produced practical attacks on the full-round constructions, indicating no practical key-recovery; the best known attacks require significantly reduced-round variants or complexity close to exhaustive search. More specifically, the best cryptanalysis efforts in PRINCE (base design for PRINCEv2) report that a single key can be recovered with a computational complexity of $2^{125.47}$ using structural linear relations; in the related key setting, the memory complexity is 2^{33} and the time complexity 2^{64} ; using the related key boomerang attack, the complexity is 2^{39} for both memory and time [51]. The authors of PRINCEv2 claim that there is no attack against PRINCEv2 with memory complexity below 2^{47} (chosen) plaintext-ciphertext pairs (obtained

TABLE V: Modifier in representative PAC-based defenses.

Work	Modifier Design
PARTS [62]	Stack pointer (SP) for return addresses, and a type identifier for indirect and data pointers.
PACStack [61]	Previous return address on the stack.
PTAuth [40]	A generated object-id.
PACSan / PACMem [59], [60]	A static random number generated at compile time.
AOS [53]	Stack pointer (SP) for return addresses.
PACTight [49]	Pointer location and a random tag for sensitive data pointers; previous return address and unique function ID for return addresses.
RSTI [48]	Unique mixture of pointer scope, type, permission, and location information.

under the same key) and time-complexity below 2^{112} [24], which is in line with the NIST requirement on the security of lightweight ciphers [71].

4) *Integrity with Encryption*: Theorem 1 shows that the probability of an adversary successfully forging a pointer for a target memory location p_a is no larger than breaking the cipher. Here, we discuss the probability of an attacker forging a pointer that can be unsealed successfully in the address space of A bits, but not necessarily point to a target address. As shown in 2b, after decryption, if the unused pointer bits do not match m_1 , the engine will detect that the encrypted pointer is not valid and raise an exception. Due to the confusion and diffusion property of the block cipher, the detection probability is $1 - 2^{-|m_1|}$, which is at the same security level as PAC. Even if the modifier matches, the pointer will point to a random place in the address space not controlled by the attacker, while with PAC the attacker can forge a pointer to an attacker-chosen address.

5) *Number of Modifier Bits Needed in Practice*: Given that the security will depend on the number of bits in the modifier, here we study the entropy needed in the modifier in practice.

Different pointer-protection schemes employ different modifier types, as summarized in Table V. Apple’s PAC [6] uses a zero modifier for function and vtable pointers, requiring only a single unique value. In general, the number of modifier bits required is proportional to the number of distinct contexts (modifier values) that must be represented.

PARTS-CFI [62] uses a 64-bit type identifier, but the effective entropy needed depends on the number of distinct pointer types and variables in the program (e.g., `int*`, `char*`, etc.) For example, in `xalancbm`, one of the largest benchmarks in both SPEC CPU2006 and SPEC CPU2017, there are 2,558 pointer types [48], corresponding to roughly 12 bits of modifier space to ensure uniqueness. The same reasoning extends to other PAC-based defenses. RSTI [48] derives modifiers from pointer scope, type, permission, and location, increasing the unique context count to 14,073 for `xalancbm` and requiring approximately 14 bits; assigning a distinct modifier per pointer variable would require 16 bits. Since this benchmark represents the largest pointer footprint

across the SPEC suites, we conclude that a 16-bit modifier space is sufficient for most realistic workloads.

Since the use of m_2 reduces the maximum number of unique domains, once we decide the $|m|$ based on the required entropy for context, we use all $|m_1|_{max}$ for m_1 , and the remaining entropy is assigned to m_2 .

F. Memory Tagging and Address Width Scaling

Modern architectural trends, such as Memory Tagging Extensions (MTE) and the expansion of Virtual Address (VA) widths, significantly constrain the available non-canonical bits within a 64-bit pointer. Memory Tagging associates memory regions with metadata tags stored in the upper pointer bits to detect spatial and temporal violations. Simultaneously, scaling the address width directly reduces the unused bits previously available for in-pointer security metadata.

LIPPEN is designed to be adaptive to these architectural shifts. Since our encryption operates transparently on the pointer value, it preserves the bits reserved by hardware for addressing or tagging. For example, one can first add the memory tag to the raw pointer, and then seal the pointer with LIPPEN. At pointer dereferencing, the pointer is first decrypted and then checked for the tag. However, as the address space A grows or the tag field $|tags|$ expands, the bit-budget for modifier m_1 is proportionally reduced. To maintain a constant security margin, LIPPEN can meet the entropy requirements by using secondary modifier m_2 .

Under these constraints, the number of supported distinct security domains is bounded by the remaining key-separation entropy. Specifically, the effective domain-separation space becomes $E = 128 - |m| - |Tag| + (64 - A)$. For example, with $|m| = 16$ and $|Tag| = 4$: (i) if $A = 48$, then $E = 124$ bits (2^{124} domains); (ii) if $A = 57$ (x86_64 servers with 5-level page tables), then $E = 115$ bits (2^{115} domains). They are still large enough to support the number of domains needed.

G. Discussion on Speculative Execution and Performance

Pointer encryption serializes decryption with pointer dereference, introducing non-zero latency overhead on every protected access. PARTS [62] quantifies this at 4 cycles per dereference, yielding $< 0.5\%$ overhead for code pointers but $\sim 20\%$ for all data pointers in `nbench`. We corroborate this effect using a pointer-chasing microbenchmark that measures the cost of accessing signed pointers on the Apple M1 processor. Although such overheads may be acceptable for PAC-style deployments, they motivate architectural optimizations to reduce dereference latency. Prior designs like C3 [58] discuss the optimizations like predictions, showing close to zero protection overhead. But C3 design is based on Intel architectures, limiting some optimization.

Code pointers are dereferenced in branch, jump, or return instructions. The branch predictor will still work as is. Branch target prediction like branch target buffer (BTB) and return address stack (RAS) usually use the PC of the current branch instruction for prediction instead of the pointer itself. Pointer encryption does not touch the predictor design, and thus,

the prediction rate of the branch target will not be affected. With pointer encryption, the resolution of the branch will take one more cycle, which adds to the execution latency when a branch misprediction happens and has negligible overhead on a correct branch prediction. With a decent branch prediction, the performance overhead will be small, as shown in designs like PAC. We provide further proof of the effects of BTB and RAS on overhead in section VI.

V. IMPLEMENTATION

a) Hardware: We prototype LIPPEN on a 64-bit RISC-V platform implemented on an AMD/Xilinx VCU118 (Virtex UltraScale+) FPGA booting a FireMarshal-managed Linux image [72]. The system is built using Chipyard [4] v1.8 and synthesized with Xilinx Vivado 2021.2. Our design extends both the in-order Rocket and out-of-order BOOM cores [14] with a custom cryptographic accelerator for PRINCEv2 connected via the Rocket Custom Coprocessor (RoCC) interface. The accelerator operates on 64-bit pointers and communicates with the core through tightly coupled request and response queues. We also implement QARMA in RoCC on the FPGA platform to use as an authentication-based PAC baseline. We configure LIPPEN with $M_1 = 16$ bits and $M_2 = 0$, as 16 bits suffice to uniquely distinguish all pointer contexts. Our Rocket and Large Boom cores are configured with default settings.

b) Compiler Support: For return address protection, we use the stack pointer as the modifier input, which is similar to RETAA for PAC, and our compiler support is implemented in LLVM [56] v18.1 by extending the RISC-V backend to instrument call and return sites with LIPPEN sealing and unsealing instructions. To demonstrate compatibility with prior PAC-based protection (G6), we leverage PacTight [49] and its LLVM-based compiler framework, making minor modifications to its IR pass to target the RISC-V architecture.

VI. EVALUATION

A. Evaluation Method

Evaluation Setup. We evaluate on both Rocket (in-order) and BOOM (out-of-order) cores on our FPGA platform running at 100 MHz. LIPPEN provides pointer protection via PRINCEv2-based encryption, while our PAC design does authentication using our QARMA implementation; we additionally compare against Apple’s PAC on the commercial M1 processor.

Performance Overhead. We evaluate runtime overhead and the number of instructions retired relative to uninstrumented baselines across our benchmark suites. Our timings capture the execution cost introduced by the added instructions (e.g., fetching, decoding in the pipeline) and pointer sealing and unsealing operations in LIPPEN. Each experiment is repeated at least twice to ensure it is not affected by significant noise.

Compatibility. We adopt PacTight [49] from Arm PAC to LIPPEN in RISC-V, to evaluate the migration complexity.

Hardware Cost. We report FPGA resource utilization (LUTs and flip-flops) for our Rocket and BOOM core with

and without the encryption accelerator. This metric evaluates the hardware footprint of the pointer encryption logic.

Power Consumption. Power estimates are obtained from post-synthesis analysis on the VCU118 board. This metric complements area evaluation and quantifies the energy efficiency of the encryption accelerator.

B. Benchmarks

Micro-benchmarks. We design a set of targeted microbenchmarks to isolate the runtime overheads of data pointer and return address protection.

Return address protection. To isolate signing and authentication costs under different call-stack behaviors, we design four microbenchmarks: (1) Looped nested function calls: a loop of nested function calls with a depth of 8 to represent common user programs with nested function calls. (2) Looped function calls: a function repeatedly invoked in a tight loop. We implement two variants: `function_S`, containing a single xor, and `function_L`, containing three xors and one memory access. Timing measurements include both signing and authentication, capturing the combined per-call overhead. (3) Deep Recursive entry-only: a very deep recursive function (depth=4096) that measures only function entry (and signing). (4) Deep Recursive return-only: a very deep recursive function (depth=4096) that measures only function return (and authentication). All benchmarks are automatically instrumented: on RISC-V using our LLVM-based compiler pass, and on Arm using the `arm64e` compilation flag. Together, these microbenchmarks measure the performance of function entry and return in different return address prediction scenarios, enabling precise characterization of LIPPEN’s protection overhead.

Data pointer protection. We implement a pointer-chasing benchmark that forms a strictly serialized load chain, ensuring authentication lies on the critical path. Two variants are evaluated: (1) *Loop*, with a single dependent load per iteration, and (2) *Unrolled*, with 32 dependent loads per iteration to amortize the impact of loop instructions. For each, we test authentication with (a) zero modifier, without the need to load the modifier to the register, (b) a shared non-zero modifier for all accesses per iteration, which loads the modifier once per loop iteration, and (c) a non-zero modifier loaded per access. We manually insert load and authentication instructions at the assembly level to ensure precise control over the execution sequence. We additionally evaluate Apple M1’s fused load-and-authenticate instruction. These configurations isolate intrinsic authentication latency, modifier-fetch overhead, and fusion benefits.

End-to-end evaluation. We evaluate full benchmarks with return address protection applied automatically using our LLVM compiler pass. We first evaluate LIPPEN using the `nbench` suite to characterize overhead on lightweight compute kernels. These benchmarks stress arithmetic and memory subsystems in isolation, enabling controlled measurement of pointer-protection cost without full-application complexity.

We then evaluate LIPPEN on the SPEC CPU2017 rate suite, covering diverse compute- and memory-intensive workloads

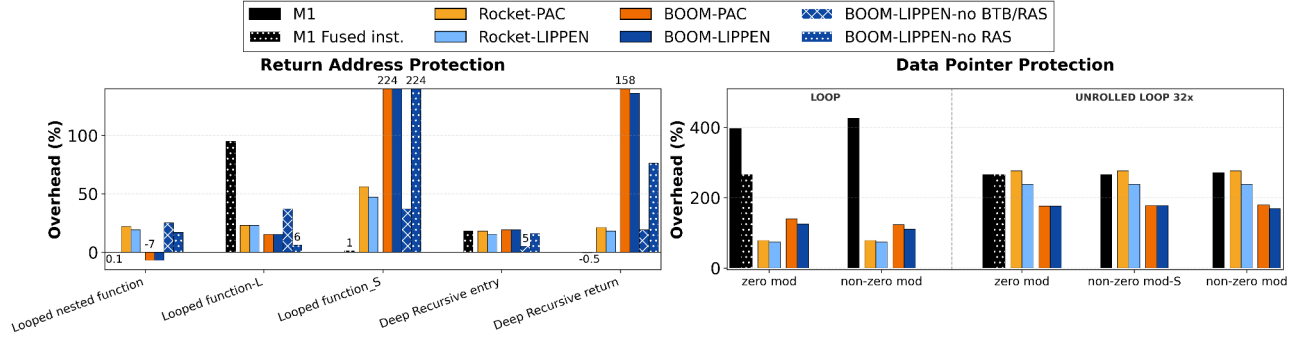


Fig. 3: Microbenchmark results for return-address (left) and data-pointer (right) protection. Runtime slowdowns are normalized to the unprotected baseline on the same processor and configuration.

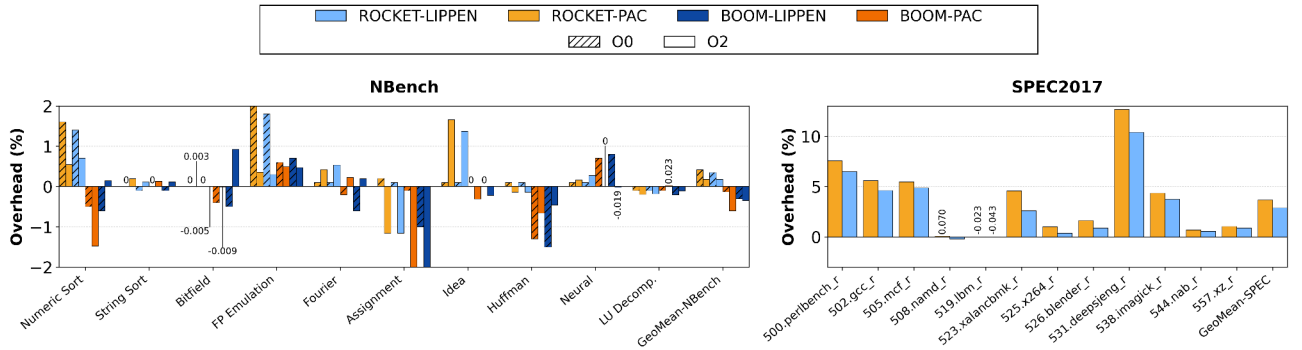


Fig. 4: Overhead across NBench (left) and SPEC CPU2017 (right). Bars show runtime overhead, all normalized to the corresponding unprotected baseline on the same processor.

representative of modern systems. Our study includes C and C++ benchmarks compiled with LLVM-based instrumentation, spanning integer workloads (e.g., `perlbenc_r`, `gcc_r`, `xalanbm_r`) and floating-point workloads (e.g., `lbm_r`, `namd_r`), thereby capturing varied control-flow and memory-access behaviors. Certain SPEC benchmarks are excluded due to interactions with the C++ exception-handling runtime. During stack unwinding, sealed pointers may be dereferenced without prior unsealing, causing incorrect control flow. Supporting this would require modifications to system libraries (e.g., recompiling GCC runtime components to insert unsealing), which is orthogonal to LIPPEN’s architectural design and left to future work.

C. Performance Results

Micro-benchmark results.

(a) *Return address protection.* The left side of Figure 3 reports the overhead of return-address protection. Overall, LIPPEN on BOOM is comparable to Apple’s M1 in most cases, and LIPPEN has a slightly smaller overhead than PAC on the FPGA platforms. The dominant factor influencing performance is return prediction rather than the protection primitive itself. For example, in the looped nested-function call benchmark, both BOOM and Apple M1 show negligible

overhead or even small performance improvements. Disabling the RAS on BOOM significantly increases overhead. For looped single function calls, the overhead varies across architectures and function body sizes. The extremely small function body (looped function-S) increases misprediction frequency on BOOM, leading to substantially higher overhead compared to looped function-L, while M1 has the opposite behavior. For deep recursive returns, where the BOOM RAS capacity is exceeded, disabling both the RAS and BTB on BOOM can reduce overhead, as repeated mispredictions and recovery penalties amplify authentication latency. Meanwhile, Apple M1 shows negligible overhead because M1’s predictor can still make correct return predictions. Deep recursion entry operations, however, show relatively low sensitivity to prediction structures, since signing occurs prior to control-flow resolution and Rocket, BOOM, and M1 show similar overheads. When prediction mechanisms are effective, BOOM achieves overhead comparable to commercial M1 implementations while preserving the protection guarantees of LIPPEN.

(b) *Data pointer protection.* The right side of Figure 3 reports the overhead of data-pointer protection due to authentication. The overhead of LIPPEN in BOOM is comparable to that of Apple’s M1. On FPGA, LIPPEN consistently tracks QARMA-based PAC implementations. Interestingly, introduc-

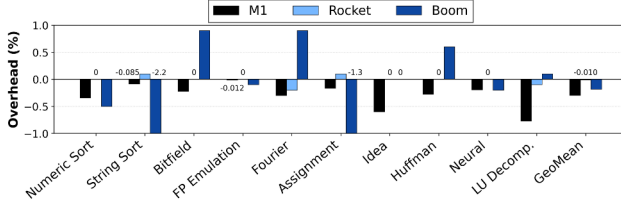


Fig. 5: Performance comparison of PacTight on different cores.

ing an additional load for the modifier does not significantly affect performance across architectures. Although a non-zero modifier requires additional load to fetch its value, this load is typically independent of the critical load-use chain and can overlap with other in-flight operations and in our benchmark the load of modifier will always hit in L1. Finally, we attribute the difference between looped and unrolled $32\times$ configurations to the differences in how the pipelines handle loops, e.g., branch prediction. The unrolled $32\times$ amortized the effect of loops. These results demonstrate that full pointer encryption is not more expensive than pointer authentication in commercial processors while providing a much stronger security level.

End-to-end evaluation results.

Figure 4 reports the runtime overhead of LIPPEN relative to PAC across nbench and SPEC2017, normalized to the uninstrumented baseline. On nbench, both LIPPEN and PAC incur only 0.2% additional number of retired instructions and a 0.2% geometric mean overhead on Rocket under O2 optimization, while BOOM shows a small performance improvement. Under O0, overheads remain comparably low, with geometric means of 0.35% and 0.42% for LIPPEN and PAC on Rocket, and near-zero on BOOM. These results are consistent with prior reports for return-address protection on nbench (e.g., 0.5% in PARTS [62] and 0.11% in Rettag [91]), confirming negligible cost when call density is low. Across SPEC on Rocket, return-address protection increases the number of retired instructions by 1% on average, yielding geometric mean overheads of 2.9% for LIPPEN and 3.6% for PAC. Control-flow-intensive workloads (e.g., perlbench_r, deepsjeng_r) exhibit higher overheads (6–12%), whereas compute- and memory-bound applications (e.g., namd_r, lbm_r, nab_r) remain near baseline. Overall, LIPPEN matches and slightly outperforms PAC while providing stronger integrity and confidentiality guarantees. Overheads remain modest and are driven primarily by control-flow intensity rather than by full-pointer encryption.

D. Compatibility with Prior Work

PacTight [49] enforces pointer integrity using strong, unique modifiers to protect sensitive pointers and provide spatial and temporal memory safety. It instruments programs via an LLVM IR pass that inserts PAC signing and authentication instructions. To evaluate compatibility, we integrated LIPPEN into PacTight’s compilation flow by replacing the original Arm pacia and authia instructions with our RISC-V seal and unseal instructions. This required only minor modifications to the IR pass (fewer than 50 lines of code),

TABLE VI: FPGA area and power comparison on Rocket.

Config	Chip		Core		ROCC		Max Freq. (MHz)	Power (W)
	LUT	FF	LUT	FF	LUT	FF		
Rocket-base	56,311	41,856	28,942	14,570	—	—	150	3.935
Rocket-RoCC	56,498	41,911	29,120	14,607	47	3	110	3.934
Rocket-LIPPEN	58,137	42,100	30,837	14,793	1,034	131	99	4.035
Rocket-PAC	58,519	42,110	31,212	14,791	2,071	132	89	4.031
BOOM-base	241,340	97,455	227,187	91,609	—	—	93	5.389
BOOM-RoCC	248,914	98,245	234,783	92,392	48	3	86.5	5.532
BOOM-LIPPEN	248,416	98,674	234,289	92,819	862	193	90.6	5.625
BOOM-PAC	251,299	97,919	236,953	92,074	2,054	131	73.5	5.606

indicating minimal integration effort and no structural changes to the underlying protection model. While small ISA-specific differences exist in the IR representation between Arm and RISC-V, the resulting binaries are instrumented equivalently to the original PacTight design (Complete similarity in the PacTight’s provided example code and nbench). As presented in Figure 5, compiling and running nbench with the adapted PacTight infrastructure on Apple M1, Rocket, and BOOM results in negligible performance overhead under $-O0$, typically under 1%, with Apple M1 and BOOM consistently exhibiting slight performance improvements. Under $-O2$, M1 overhead remains near-zero, while Rocket and BOOM increase to 7.50% and 2.88%, respectively. We attribute this to the compiler treating LIPPEN instructions as opaque barriers, preventing code motion and scheduling optimizations around them. These results demonstrate that LIPPEN composes cleanly with prior PAC-based defenses while preserving their low cost profile across different microarchitectures.

E. Hardware Cost and Power Results

Table VI summarizes FPGA resource utilization, maximum frequency, and power (measured at 100MHz) across all configurations on both cores. Rocket-RoCC and BOOM-RoCC instantiate the ROCC interface without any cipher, revealing the overhead due to the ROCC interface. Rocket-LIPPEN and BOOM-LIPPEN add the PRINCEv2-based encryption accelerator, and Rocket-PAC and BOOM-PAC add the QARMA-based authentication accelerator. For both cores, the QARMA configuration incurs slightly more logic than PRINCEv2 due to its more complex cipher datapath. Power overhead remains marginal across all configurations on both cores. Overall, LIPPEN achieves full-pointer encryption at hardware and power costs commensurate with cipher complexity, confirming its practicality for integration into modern processor pipelines.

VII. DISCUSSION

A. Secure Key Management

Secure key management is essential to prevent key exposure and cross-domain forgery. While Arm Pointer Authentication (PA) provides hardware key registers [9], sharing keys across exception levels can enable cross-domain attacks [27]. Recent designs, such as Apple’s M-series processors, address this via

per-VM, per-EL, and per-boot key isolation using hardware-backed diversification and internal key derivation [27]. LIPPEN can use the same mechanisms. Because sealing and unsealing are keyed primitives analogous to PAC instructions, domain-specific keys and hierarchical derivation apply directly. Thus, PAC-style key isolation extends to LIPPEN without architectural changes, keeping key management orthogonal to the encryption mechanism while leveraging existing secure hardware deployments.

B. Pointer Arithmetic

A potential concern when protecting pointers is the cost of pointer arithmetic. In C and C++, pointers can be incremented or adjusted (e.g., during array traversal), and both Arm PAC and LIPPEN will complicate the arithmetic by requiring an authentication/decryption before the pointer arithmetic. However, pointer arithmetic without a subsequent memory access is rare in practice. Most arithmetic operations on pointers are immediately followed by pointer dereference, and one can optimize the order of decryption and arithmetic (in the compiler). Implementations of prior PAC-based systems that protect all pointers, such as RSTI [48] and AOS [53], show that pointer arithmetic is not the main bottleneck, and the main bottleneck is still the authentication (decryption) itself.

VIII. RELATED WORK

We have so far discussed mitigation techniques that ensure pointer integrity, focusing primarily on approaches with zero memory footprint and straightforward deployability on existing hardware. In this section, we broaden the discussion to include other methods that aim to provide general memory safety.

A. Capability and Tagged Architectures

Capability-based and tagged architectures associate each pointer with hardware-managed metadata encoding bounds, permissions, and provenance, validated on every memory access. CHERI [92] is the most prominent example, providing strong spatial and temporal memory safety at the cost of expanded pointer representations and significant ISA and ABI changes that complicate deployment within existing software ecosystems.

Secure tagged architectures [43] similarly attach metadata tags to memory and registers to enforce security policies at fine granularity. These tags can represent pointer authenticity, data provenance, or privilege levels, and are checked dynamically during instruction execution. ZeRØ [94] introduces new memory instructions and a metadata *encoding* scheme (e.g., type bits in pointers plus per-line bit-vectors) so that pointer locations can only be written by authorized instructions; invalid writes are rejected, allowing resilient operation under attack. Tag-based enforcement provides flexibility but incurs additional storage and lookup overheads and often depends on compiler or OS support to manage tag propagation.

B. Memory Safety

A complementary class of defenses targets memory safety rather than pointer integrity [11], [12], [26], [36], [40], [53], [59], [63], [78], [79], [81], [93]. These schemes associate additional metadata with memory regions to detect spatial and temporal errors such as buffer overflows and use-after-free vulnerabilities. Hardware memory tagging schemes, such as Arm’s Memory Tagging Extension (MTE) [11] and SPARC ADI [12], assign small tags (typically 4–8 bits) to memory blocks and to pointers. Each memory access compares the pointer’s tag with the memory’s tag, triggering an exception on mismatch. Software-based approaches [63], [81] implement a similar scheme in software with compiler instrumentation.

A recent line of work leverages pointer authentication for enforcing spatial and temporal memory safety [40], [53], [59]. They build upon Arm Pointer Authentication Codes (PAC) to cryptographically bind pointers to their allocation or lifetime metadata, thereby preventing out-of-bounds and use-after-free violations.

Other proposals extend tagging granularity or functionality. Califorms [78] introduces fine-grained, byte-level tagging to blacklist invalid memory regions, while MEMES [79] employs lightweight encryption to enforce spatial and temporal safety on commodity hardware. Similarly, systems such as CUP [26], HardBound [36], CAMP [63], and No-FAT [93] combine compiler or architectural support with metadata tracking to enforce object and bounds checking. These designs strengthen full memory safety but maintain separate metadata storage or architectural state, resulting in additional memory footprint and runtime overhead.

IX. CONCLUSION

This paper presented LIPPEN, a low-overhead pointer-encryption architecture that eliminates the limitations of authentication-based pointer integrity enforcement through full-pointer encryption. By encrypting the entire pointer value rather than storing a truncated authentication code, LIPPEN maximizes entropy, achieves brute-force resilience, and unifies protection for both code and data pointers. The design maintains seamless compatibility with the existing PAC software stack and ABI, requiring no changes to compilers or operating systems. Our prototype on a RISC-V FPGA platform protects return addresses and demonstrates that LIPPEN delivers strong pointer integrity with cost comparable to PAC and minimal hardware and power increases. These results show that full-pointer encryption is both practical and effective, offering cryptographic-level protection against pointer forgery while remaining suitable for deployment in modern processors.

X. ACKNOWLEDGMENT

At Virginia Tech, this project is partially supported by Commonwealth Cybersecurity Initiative (CCI), by the National Science Foundation (NSF) under grant CCF-2153748, CNS-2442993. LLMs were used for editorial purposes, with all outputs inspected by the authors to ensure accuracy and originality.

A. Abstract

The artifact is designed to enable reproduction of the core components and a representative subset of the evaluation results, while supporting full reproduction given additional hardware and setup time.

The artifact guides users through compiling and simulating the Chipyard design with Verilator and running small test programs. It also includes instructions for generating a bitstream for the FPGA implementation. To run the microbenchmarks on our prototype, users need access to a Xilinx VCU118 FPGA and must follow the documented steps for building the Linux image, preparing the SD card with the required RISC-V binaries, and transferring files through the SD card workflow. In addition, the artifact includes microbenchmarks for ARM64 processors, which we tested on Apple M1 systems and expect to work on other Apple M-series processors as well. Because Apple restricts the use of PAC instructions in user-space programs, users must disable System Integrity Protection before running those experiments.

B. Artifact check-list (meta-information)

- **Compilation:** Compiling the modified Chipyard hardware design, the LLVM-based compiler toolchain, and the provided microbenchmarks.
- **Run-time environment:** Ubuntu Linux for building and simulation; firemarshal linux for running on FPGA; macOS for ARM64 microbenchmark experiments.
- **Hardware:** A server or workstation for building and simulation; a Xilinx VCU118 FPGA for FPGA-based experiments; an Apple M1-based machine for ARM64 microbenchmark evaluation.
- **Execution:** Running Verilator-based simulations, generating FPGA bitstreams, booting Linux on the FPGA prototype, and executing the provided microbenchmarks both on FPGA and Apple M1.
- **Output:** Generated simulation binaries, FPGA bitstreams, compiled benchmark binaries, and performance measurement results.
- **How much time is needed to prepare workflow (approximately)?:** Around 30–60 minutes for software and simulation setup; the FPGA synthesis and bitstream generation take hours.
- **Publicly available?:** Yes, artifacts can be found here: <https://doi.org/10.5281/zenodo.19901476>
<https://github.com/bearhw/LIPPEN>
- **Code licenses (if publicly available)?:** GNU General Public License v3.0.

C. Description

1) *How to access:* The artifact can be accessed here <https://doi.org/10.5281/zenodo.19901476>. It can also be accessed through the public GitHub repository by cloning the repository <https://github.com/bearhw/LIPPEN>. The README file in the repository contains the full instructions for building and running the artifact.

2) *Hardware dependencies:* The artifact supports multiple workflows with different hardware requirements. For simulation and software compilation, a standard Linux server or workstation is sufficient. For FPGA-based evaluation, the artifact requires a Xilinx VCU118 FPGA board. For the ARM64 microbenchmark experiments, the artifact requires an Apple M1 system, and it should also work on other Apple M-series processors.

3) *Software dependencies:* The artifact requires a Linux environment with the dependencies needed to build the modified Chipyard design, the LLVM-based compiler toolchain, and the provided benchmarks. Running the FPGA workflow additionally requires the tools needed to generate the bitstream, build the Linux image, prepare the SD card, and execute binaries on the prototype. For the ARM64 microbenchmark workflow, the artifact requires macOS with the appropriate build tools installed. Because Apple restricts the use of PAC instructions in user-space programs, System Integrity Protection must be disabled before running those experiments.

D. Installation

Clone the GitHub repository. Then follow the instructions in the README file to set up the build environment, compile the required components, and prepare the selected workflow. The README describes separate steps for simulation, FPGA-based evaluation, and ARM64 microbenchmark experiments.

E. Experiment workflow

- 1) Clone the repository and initialize the required submodules and dependencies.
- 2) Build the modified Chipyard hardware design and the LLVM-based compiler toolchain by following the provided scripts.
- 3) Run the Verilator-based simulation flow to verify the design and execute the included small test programs.
- 4) If FPGA evaluation is desired, generate the FPGA bitstream and prepare the Linux image for the VCU118 platform.
- 5) Load the required binaries and files onto the SD card and boot the system on the FPGA prototype.
- 6) Execute the provided microbenchmarks and collect the performance results.
- 7) For the ARM64 workflow, compile and run the microbenchmarks on an Apple M1 or another Apple M-series processor after disabling System Integrity Protection.

F. Evaluation and expected results

Successful execution of the artifact is demonstrated by:

- 1) Correctly building the modified Chipyard design and LLVM-based toolchain.
- 2) Running the provided test programs in the Verilator simulation.
- 3) Reproducing the experimental results from Figure 4 of the paper.

The FPGA-based experiments require access to a Xilinx VCU118 board and involve significant synthesis and setup time. Similarly, the ARM64 microbenchmark experiments require Apple M-series hardware and additional system configuration (e.g., disabling System Integrity Protection).

REFERENCES

- [1] “arm64: ptrauth: add pointer authentication Armv8.6 enhanced feature,” <http://www.spinics.net/lists/arm-kernel/msg814954.html>. Accessed nov 2025.
- [2] M. Abadi, M. Budiu, U. Erlingsson, and J. Ligatti, “Control-flow integrity principles, implementations, and applications,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 13, no. 1, pp. 1–40, 2009.
- [3] F. Abed, E. List, and S. Lucks, “On the security of the core of PRINCE against biclique and differential cryptanalysis,” *Cryptology ePrint Archive*, Paper 2012/712, 2012. [Online]. Available: <https://eprint.iacr.org/2012/712>
- [4] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton *et al.*, “Chipyard: Integrated design, simulation, and implementation framework for custom socs,” *Ieee Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [5] Apple, “Preparing your app to work with pointer authentication,” <https://developer.apple.com/documentation/security/preparing-your-app-to-work-with-pointer-authentication>. Accessed nov 2025.
- [6] —, “Operating system integrity,” 2021, <https://support.apple.com/en-hk/guide/security/sec8b776536b1/web>.
- [7] Apple Inc. (2023) Pointer authentication in dyld. Accessed: 2025-02-05. [Online]. Available: <https://deepwiki.com/apple-oss-distributions/dyld/6.2-pointer-authentication>
- [8] ARM, “Rop,” ARM, Tech. Rep., 2018, available at: <https://developer.arm.com/documentation/102433/0200/Return-oriented-programming>.
- [9] Arm Limited, *Arm Architecture Reference Manual for A-profile architecture*, rev. k.a ed., Arm Limited, 2023, document ARM DDI 0487. [Online]. Available: <https://developer.arm.com/documentation/ddi0487/latest/>
- [10] *Architecture Extensions before 2020: Speculative behavior of pointer authentication instructions (FEAT_FPACC_SPEC)*, Arm Limited, 2024, version 1.2. [Online]. Available: <https://developer.arm.com/documentation/110389/1-2/>
- [11] Arm Ltd., “Arm architecture reference manual supplement: Memory tagging extension,” https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Arm_Memory_Tagging_Extension_Whitepaper.pdf, 2019, accessed: 2025-11-03.
- [12] —, “Hardware-assisted checking using silicon secured memory (ssm),” https://docs.oracle.com/cd/E37069_01/html/E37085/gphwb.html, 2019, accessed: 2025-11-03.
- [13] *Arm® Architecture Reference Manual Armv8, for Armv8-A architecture profile*, Version i.a ed., Arm Ltd., 2024, [Online; accessed Nov. 16, 2025]. [Online]. Available: <https://developer.arm.com/documentation/110389/latest/>
- [14] K. Asanovic, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz *et al.*, “The rocket chip generator,” *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17*, vol. 4, pp. 6–2, 2016.
- [15] C. Auth, A. Aliyarukunju, M. Asoro, D. Bergstrom, V. Bhagwat, J. Birdsall, N. Bisnik, M. Buehler, V. Chikarmane, G. Ding, Q. Fu, H. Gomez, W. Han, D. Hanken, M. Haran, M. Hattendorf, R. Heussner, H. Hiramatsu, B. Ho, S. Jaloviar, I. Jin, S. Joshi, S. Kirby, S. Kosaraju, H. Kothari, G. Leatherman, K. Lee, J. Leib, A. Madhavan, K. Marla, H. Meyer, T. Mule, C. Parker, S. Parthasarathy, C. Peltó, L. Pipes, I. Post, M. Prince, A. Rahman, S. Rajamani, A. Saha, J. D. Santos, M. Sharma, V. Sharma, P. Shin, P. Sinha, P. Smith, M. Sprinkle, A. S. Amour, C. Staus, R. Suri, D. Towner, A. Tripathi, A. Tura, C. Ward, and A. Yeoh, “A 10nm high performance and low-power CMOS technology featuring 3rd generation FinFET transistors, Self-Aligned Quad Patterning, contact over active gate and cobalt local interconnects,” in *2017 IEEE International Electron Devices Meeting (IEDM)*, 2017, pp. 29.1.1–29.1.4.
- [16] R. Avanzi, “The qarma block cipher family. almost mds matrices over rings with zero divisors, nearly symmetric even-mansour constructions with non-involutory central rounds, and search heuristics for low-latency s-boxes,” *IACR Transactions on Symmetric Cryptology*, pp. 4–44, 2017.
- [17] R. Avanzi, S. Banik, O. Dunkelman, M. Eichlseder, S. Ghosh, M. Nageler, and F. Regazzoni, “The QARMav2 Family of Tweakable Block Ciphers,” *IACR Transactions on Symmetric Cryptology*, vol. 2023, no. 3, p. 25–73, Sep. 2023. [Online]. Available: <https://tosc.iacr.org/index.php/ToSC/article/view/11184>
- [18] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, “The SIMON and SPECK Lightweight Block Ciphers,” in *Proceedings of the 52nd Annual Design Automation Conference*, ser. DAC ’15. New York, NY, USA: Association for Computing Machinery, 2015. [Online]. Available: <https://doi.org/10.1145/2744769.2747946>
- [19] Y. Belkheyar, J. Daemen, C. Dobraunig, S. Ghosh, and S. Rasoolzadeh, “BipBip: A Low-Latency Tweakable Block Cipher with Small Dimensions,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2023, no. 1, p. 326–368, Nov. 2022. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/9955>
- [20] B. Bierbaumer, J. Kirsch, T. Kittel, A. Francillon, and A. Zarras, “Smashing the stack protector for fun and profit,” in *IFIP International Conference on ICT Systems Security and Privacy Protection*. Springer, 2018, pp. 293–306.
- [21] T. Bletsch, X. Jiang, V. W. Freeh, and Z. Liang, “Jump-oriented programming: a new class of code-reuse attack,” in *Proceedings of the 6th ACM symposium on information, computer and communications security*, 2011, pp. 30–40.
- [22] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsøe, “PRESENT: An Ultra-Lightweight Block Cipher,” in *Cryptographic Hardware and Embedded Systems - CHES 2007*, P. Paillier and I. Verbauwhede, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 450–466.
- [23] J. Borghoff, A. Canteaut, T. Güneysu, E. B. Kavun, M. Knezevic, L. R. Knudsen, G. Leander, V. Nikov, C. Paar, C. Rechberger, P. Rombouts, S. S. Thomsen, and T. Yalçın, “PRINCE: a low-latency block cipher for pervasive computing applications,” in *Proceedings of the 18th International Conference on The Theory and Application of Cryptology and Information Security*, ser. ASIACRYPT’12. Berlin, Heidelberg: Springer-Verlag, 2012, p. 208–225. [Online]. Available: https://doi.org/10.1007/978-3-642-34961-4_14
- [24] D. Božilov, M. Eichlseder, M. Knežević, B. Lambin, G. Leander, T. Moos, V. Nikov, S. Rasoolzadeh, Y. Todo, and F. Wiemer, “PRINCEv2: More security for (almost) no overhead,” in *Selected Areas in Cryptography: 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 483–511. [Online]. Available: https://doi.org/10.1007/978-3-030-81652-0_19
- [25] D. Brash, “Armv8-a architecture 2016 additions,” ARM Community Blog, 2016, <https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/armv8-a-architecture-2016-additions>.
- [26] N. Burow, D. McKee, S. A. Carr, and M. Payer, “Cup: Comprehensive user-space protection for c/c++,” in *Proceedings of the 2018 on Asia Conference on Computer and Communications Security*, 2018, pp. 381–392.
- [27] Z. Cai, J. Zhu, W. Shen, Y. Yang, R. Chang, Y. Wang, J. Li, and K. Ren, “Demystifying pointer authentication on apple m1,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 2833–2848.
- [28] C. Canière, O. Dunkelman, and M. Knežević, “KATAN and KTANTAN – A Family of Small and Efficient Hardware-Oriented Block Ciphers,” in *Proceedings of the 11th International Workshop on Cryptographic Hardware and Embedded Systems*, ser. CHES ’09. Berlin, Heidelberg: Springer-Verlag, 2009, p. 272–288. [Online]. Available: https://doi.org/10.1007/978-3-642-04138-9_20
- [29] N. Carlini, A. Barresi, M. Payer, D. Wagner, and T. R. Gross, “Control-Flow bending: On the effectiveness of Control-Flow integrity,” in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 161–176.
- [30] N. Carlini and D. Wagner, “ROP is still dangerous: Breaking modern defenses,” in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 385–399.
- [31] M. S. R. Center. (2019) A proactive approach to more secure code. Reports 70% of Microsoft CVEs are memory-safety issues. [Online]. Available: <https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code>

- [32] S. Chen, J. Xu, E. C. Sezer, P. Gauriar, and R. K. Iyer, "Non-control-data attacks are realistic threats." in *USENIX security symposium*, vol. 5, 2005, p. 146.
- [33] C. Cowan, S. Beattie, J. Johansen, and P. Wagle, "PointGuard™: Protecting pointers from buffer overflow vulnerabilities," in *12th USENIX Security Symposium (USENIX Security 03)*, 2003.
- [34] L. Davi, A.-R. Sadeghi, D. Lehmann, and F. Monrose, "Stitching the gadgets: On the ineffectiveness of Coarse-Grained Control-Flow integrity protection," in *23rd USENIX Security Symposium (USENIX Security 14)*. San Diego, CA: USENIX Association, Aug. 2014, pp. 401–416. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/davi>
- [35] P. Derbez and L. Perrin, "Meet-in-the-Middle Attacks and Structural Analysis of Round-Reduced PRINCE," *J. Cryptol.*, vol. 33, no. 3, p. 1184–1215, 2020.
- [36] J. Devietti, C. Blundell, M. M. Martin, and S. Zdancewic, "Hardbound: Architectural support for spatial safety of the c programming language," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 2, pp. 103–114, 2008.
- [37] U. Dhawan, C. Hritcu, R. Rubin, N. Vasilakis, S. Chiricescu, J. M. Smith, T. F. Knight Jr, B. C. Pierce, and A. DeHon, "Architectural support for software-defined metadata processing," in *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2015, pp. 487–502.
- [38] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  fer, "Ascon v1.2: Lightweight Authenticated Encryption and Hashing," *J. Cryptol.*, vol. 34, no. 3, Jul. 2021. [Online]. Available: <https://doi.org/10.1007/s00145-021-09398-9>
- [39] I. Evans, F. Long, U. Otgonbaatar, H. Shrobe, M. Rinard, H. Okhravi, and S. Sidiroglou-Douskos, "Control jujutsu: On the weaknesses of fine-grained control flow integrity," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 901–913.
- [40] R. M. Farkhani, M. Ahmadi, and L. Lu, "PTAuth: temporal memory safety via robust points-to authentication," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 1037–1054.
- [41] M. Gallagher, L. Biernacki, S. Chen, Z. B. Aweke, S. F. Yitbarek, M. T. Aga, A. Harris, Z. Xu, B. Kasicki, V. Bertacco *et al.*, "Morpheus: A vulnerability-tolerant secure architecture based on ensembles of moving target defenses with churn," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 469–484.
- [42] E. G  ktas, K. Razavi, G. Portokalidis, H. Bos, and C. Giuffrida, "Speculative probing: Hacking blind in the spectre era," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1871–1885.
- [43] R. T. Gollapudi, G. Yuksek, D. Demicco, M. Cole, G. Kothari, R. Kulkarni, X. Zhang, K. Ghose, A. Prakash, and Z. Umrigar, "Control flow and pointer integrity enforcement in a secure tagged architecture," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 2974–2989.
- [44] Z. Gong, S. Nikova, and Y. W. Law, "KLEIN: A New Family of Lightweight Block Ciphers," in *RFID. Security and Privacy*, A. Juels and C. Paar, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 1–18.
- [45] J. Guo, T. Peyrin, A. Poschmann, and M. Robshaw, "The LED Block Cipher," in *Cryptographic Hardware and Embedded Systems – CHES 2011*, B. Preneel and T. Takagi, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 326–341.
- [46] H. Hu, S. Shinde, S. Adrian, Z. L. Chua, P. Saxena, and Z. Liang, "Data-oriented programming: On the expressiveness of non-control data attacks," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 969–986.
- [47] Institute of Applied Information Processing and Communications (IAIK), TU Graz, "memsec: Hardware security primitives for memory protection," <https://github.com/isec-tugraz/memsec/tree/develop/hdl/crypto>, 2025, contains VHDL implementations `qarma.vhd` and `prince.vhd` accessed on November 17, 2025.
- [48] M. Ismail, C. Jelesnianski, Y. Jang, C. Min, and W. Xiong, "Enforcing c/c++ type and scope at runtime for control-flow and data-flow integrity," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 283–300.
- [49] M. Ismail, A. Quach, C. Jelesnianski, Y. Jang, and C. Min, "Tightly seal your sensitive pointers with PACTight," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3717–3734.
- [50] D. Jang, Z. Tatlock, and S. Lerner, "SafeDispatch: Securing c++ virtual calls from memory corruption attacks." in *NDSS*, 2014.
- [51] J. Jean, I. Nikoli  , T. Peyrin, L. Wang, and S. Wu, "Security analysis of PRINCE," in *Fast Software Encryption (FSE)*, ser. Lecture Notes in Computer Science, vol. 8424. Springer, 2014, pp. 92–111.
- [52] J. Kim, J. Park, S. Roh, J. Chung, Y. Lee, T. Kim, and B. Lee, "TIKTAG: Breaking arm's memory tagging extension with speculative execution," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 4063–4081.
- [53] Y. Kim, J. Lee, and H. Kim, "Hardware-based always-on heap memory safety," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 1153–1166.
- [54] V. Kiriansky and C. Waldspurger, "Speculative buffer overflows: Attacks and defenses," *arXiv preprint arXiv:1807.03757*, 2018.
- [55] M. Kounavis, S. Deutsch, S. Ghosh, and D. Durham, "K-Cipher: A Low Latency, Bit Length Parameterizable Cipher," in *2020 IEEE Symposium on Computers and Communications (ISCC)*, 2020, pp. 1–7.
- [56] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International symposium on code generation and optimization, 2004. CGO 2004*. IEEE, 2004, pp. 75–86.
- [57] T. Lelegard, "Arm system registers: PAC format," <https://github.com/lelegard/arm-cpusysregs/blob/main/docs/pac-format.md>, 2024, accessed: May 2024.
- [58] M. LeMay, J. Rakshit, S. Deutsch, D. M. Durham, S. Ghosh, A. Nori, J. Gaur, A. Weiler, S. Sultana, K. Grewal *et al.*, "Cryptographic capability computing," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 253–267.
- [59] Y. Li, W. Tan, Z. Lv, S. Yang, M. Payer, Y. Liu, and C. Zhang, "Pacmem: Enforcing spatial and temporal memory safety via arm pointer authentication," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1901–1915.
- [60] —, "Pacsan: Enforcing memory safety based on arm pa," *arXiv preprint arXiv:2202.03950*, 2022.
- [61] H. Liljestrand, T. Nyman, L. J. Gunn, J.-E. Ekberg, and N. Asokan, "PACStack: an authenticated call stack," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 357–374. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/liljestrand>
- [62] H. Liljestrand, T. Nyman, K. Wang, C. C. Perez, J.-E. Ekberg, and N. Asokan, "PAC it up: Towards pointer integrity using ARM pointer authentication," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 177–194.
- [63] Z. Lin, Z. Yu, Z. Guo, S. Campanoni, P. Dinda, and X. Xing, "CAMP: Compiler and allocator-based heap memory protection," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 4015–4032.
- [64] LLVM Project. (2024) Pointer authentication in clang/llvm. Accessed: 2025-02-05. [Online]. Available: <https://clang.llvm.org/docs/PointerAuthentication.html>
- [65] B. B. Madan, S. Phoha, and K. S. Trivedi, "StackOFFence: a technique for defending against buffer overflow attacks," in *International Conference on Information Technology: Coding and Computing (ITCC'05)-Volume II*, vol. 1. IEEE, 2005, pp. 656–661.
- [66] M. Mahzoun, L. Kraleva, R. Posteuca, and T. Ashur, "Differential Cryptanalysis of K-Cipher," in *2022 IEEE Symposium on Computers and Communications (ISCC)*, 2022, pp. 1–7.
- [67] A. J. Mashtizadeh, A. Bittau, D. Boneh, and D. Mazi  res, "CCFI: Cryptographically enforced control flow integrity," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 941–951.
- [68] C. McGarr. (2023) Windows on arm64 pointer authentication codes (pac). Accessed: 2025-02-05. [Online]. Available: <https://connormcgarr.github.io/windows-pac-arm64/>
- [69] P. Morawiecki, "Practical attacks on the round-reduced PRINCE," *Cryptology ePrint Archive, Paper 2015/245*, 2015. [Online]. Available: <https://eprint.iacr.org/2015/245>
- [70] W. T. Na, J. S. Emer, and M. Yan, "Penetrating shields: A systematic analysis of memory corruption mitigations in the spectre era," *arXiv preprint arXiv:2309.04119*, 2023.
- [71] NIST, "Submission requirements and evaluation criteria for the lightweight cryptography standardization process," 2018. [Online]. Available: <https://csrc.nist.gov/CSRC/media/Projects/Lightweight>

- Cryptography/documents/final-lwc-submission-requirements-august2018.pdf
- [72] N. Pemberton and A. Amid, “Firemarshal: Making hw/sw co-design reproducible and reliable,” in *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2021, pp. 299–309.
 - [73] S. D. Phayé, G. J. Duck, R. H. Yap, and T. E. Carlson, “Fully randomized pointers,” in *Proceedings of the 2025 ACM SIGPLAN International Symposium on Memory Management*, 2025, pp. 94–108.
 - [74] T. C. Project. (2020) Memory safety. 70% of high/critical Chrome bugs are memory-safety. [Online]. Available: <https://www.chromium.org/Home/chromium-security/memory-safety/>
 - [75] I. Qualcomm Technologies, “Pointer authentication on armv8.3-a,” Qualcomm Technologies, Inc., Tech. Rep. v7, Jan 2017. [Online]. Available: <https://www.qualcomm.com/content/dam/qcomm-martech/dm-assets/documents/pointer-auth-v7.pdf>
 - [76] J. Ravichandran, W. T. Na, J. Lang, and M. Yan, “PACMAN: attacking arm pointer authentication with speculative execution,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 685–698. [Online]. Available: <https://doi.org/10.1145/3470496.3527429>
 - [77] R. Rudd, R. Skowrya, D. Bigelow, V. Dedhia, T. Hobson, S. Crane, C. Liebchen, P. Larsen, L. Davi, M. Franz *et al.*, “Address oblivious code reuse: On the effectiveness of leakage resilient diversity,” in *NDSS*, 2017.
 - [78] H. Sasaki, M. A. Arroyo, M. T. I. Ziad, K. Bhat, K. Sinha, and S. Sethumadhavan, “Practical byte-granular memory blacklisting using califorms,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 558–571.
 - [79] D. Schrammel, S. Sultana, K. Grewal, M. LeMay, D. Durham, M. Unterguggenberger, P. Nasahl, and S. Mangard, “Memes: Memory encryption-based memory safety on commodity hardware,” in *20th International Conference on Security and Cryptography: SECRIPT 2023*. SciTePress, 2023, pp. 25–36.
 - [80] F. Schuster, T. Tendyck, C. Liebchen, L. Davi, A.-R. Sadeghi, and T. Holz, “Counterfeit object-oriented programming: On the difficulty of preventing code reuse attacks in c++ applications,” in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 745–762.
 - [81] K. Serebryany, E. Stepanov, A. Shlyapnikov, V. Tsyrlkevich, and D. Vyukov, “Memory tagging and how it improves c/c++ memory safety,” *arXiv preprint arXiv:1802.09517*, 2018.
 - [82] H. Shacham, “The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86),” in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 552–561.
 - [83] H. Shacham, M. Page, B. Pfaff, E.-J. Goh, N. Modadugu, and D. Boneh, “On the effectiveness of address-space randomization,” in *Proceedings of the 11th ACM conference on Computer and communications security*, 2004, pp. 298–307.
 - [84] —, “On the effectiveness of address-space randomization,” in *Proceedings of the 11th ACM conference on Computer and communications security*, 2004, pp. 298–307.
 - [85] K. Shibutani, T. Isobe, H. Hiwatari, A. Mitsuda, T. Akishita, and T. Shirai, “Piccolo: An Ultra-Lightweight Blockcipher,” in *Cryptographic Hardware and Embedded Systems – CHES 2011*, B. Preneel and T. Takagi, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 342–357.
 - [86] H. Soleimany, C. Blondeau, X. Yu, W. Wu, K. Nyberg, H. Zhang, L. Zhang, and Y. Wang, “Reflection cryptanalysis of PRINCE-like ciphers,” *Journal of Cryptology*, vol. 28, pp. 718–744, 2015.
 - [87] L. Song and L. Hu, “Differential fault attack on the PRINCE block cipher,” in *Lightweight Cryptography for Security and Privacy*, ser. Lecture Notes in Computer Science, vol. 8162. Springer, 2013, pp. 43–54.
 - [88] T. Suzaki, K. Minematsu, S. Morioka, and E. Kobayashi, “TWINE: A Lightweight Block Cipher for Multiple Platforms,” in *Selected Areas in Cryptography*, L. R. Knudsen and H. Wu, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 339–354.
 - [89] The Linux Kernel Community. (2024) Pointer authentication on arm64. Accessed: 2025-02-05. [Online]. Available: <https://docs.kernel.org/arch/arm64/pointer-authentication.html>
 - [90] V. van der Veen, D. Andriess, M. Stamatogiannakis, X. Chen, H. Bos, and C. Giuffrida, “The dynamics of innocent flesh on the bone: Code reuse ten years later,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1675–1689.
 - [91] Y. Wang, J. Wu, T. Yue, Z. Ning, and F. Zhang, “RetTag: Hardware-assisted return address integrity on risc-v,” in *Proceedings of the 15th European Workshop on Systems Security*, 2022, pp. 50–56.
 - [92] J. Woodruff, R. N. Watson, D. Chisnall, S. W. Moore, J. Anderson, B. Davis, B. Laurie, P. G. Neumann, R. Norton, and M. Roe, “The cheri capability model: Revisiting risc in an age of risk,” *ACM SIGARCH Computer Architecture News*, vol. 42, no. 3, pp. 457–468, 2014.
 - [93] M. T. I. Ziad, M. A. Arroyo, E. Manzhosov, R. Piersma, and S. Sethumadhavan, “No-FAT: Architectural support for low overhead memory safety checks,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 916–929.
 - [94] M. T. I. Ziad, M. A. Arroyo, E. Manzhosov, and S. Sethumadhavan, “ZeRØ: Zero-overhead resilient operation under pointer integrity attacks,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 999–1012.